

SML

Assign 3

April 2026

Instructions

- The assignment carries 7 marks. All questions and parts must be answered. The plots and final results should be included in the report. Support the plots and results with your analysis.
- Solve each question in a separate `.py` file.
- Use <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz> MNIST dataset for the first two questions. Unpack the `.npz` file using `np.load()` to obtain the training set of size 60,000 and the test set of size 10,000, and then select samples belonging to classes 0, 1, and 2 from these sets. The preprocessing remains similar to the previous two assignments: flatten the images and normalize the pixel values.
- You are allowed to use **sklearn only for implementing Lasso Regression**. For the rest of the assignment (except loading of data), you may only use **NumPy**, **Matplotlib**, or **Pandas** if needed.

Question-1

Perform the following tasks. For every plot generated in this question, comment briefly on the observed trends and behavior of the model.

- Apply PCA and reduce the dimension to $p = 10$. You can use the entire training set of these three classes to obtain the PCA matrix. For the remaining parts, use the reduced-dimension dataset. [CO2]
- For each class k , define the target vector as:

$$y_k = \begin{cases} 1 & \text{if the sample belongs to class } k \\ 0 & \text{otherwise} \end{cases}$$

- Perform ridge and lasso regression for classification. Train models for different values of $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$. Plot training MSE and **test MSE** as a function of λ . *Note: Use a logarithmic scale for the x-axis (λ).* [0.5+0.25] [CO2]
- For lasso regression, plot the number of non-zero coefficients as a function of λ (use a logarithmic scale for the x-axis). Also, plot the regularization paths for both Ridge and Lasso regression. To do this, for multiple values of λ , compute the corresponding coefficient matrix $W(\lambda)$ and plot the coefficient values against λ **for one specific class** (e.g., the weights associated with predicting class 1). In this plot, the x-axis should be λ (in log scale) and the y-axis should be the coefficient values, with one curve per feature. [0.25] [CO1]
- Now vary the model complexity (number of PCA dimensions $p = 2, 5, 10, 20, 30$). Use the value of λ that gave the lowest **test MSE** in the previous part. For each complexity, train a ridge regression model and compute training MSE and test MSE. Plot training error and **test error** vs model complexity. [0.25] [CO1]
- Report the test classification accuracy for the best model obtained from both ridge and lasso regression. [0.25]

Question 2

Decision Trees [1.5]

- Apply PCA and reduce the dimension to $p = 10$. You can use the entire train set of these 3 classes to obtain PCA matrix. For the remaining parts, use the reduced dimension dataset.
- Now learn a decision tree using the train set. You need to grow a decision tree with 3 terminal nodes. For the first split, consider all p dimensions as candidate splitting features. For each dimension, choose a suitable threshold value to split the data (for example mean, median, mode, or any other reasonable choice) and provide justification for your choice in the report. Compute the **weighted Gini index** for each candidate split and select the split that minimizes it. [CO2]

$$Gini(S) = 1 - \sum_{k=1}^K p_k^2$$

$$Gini_{\text{weighted}} = \sum_{i=1}^m \frac{|S_i|}{|S|} Gini(S_i)$$

where:

- $|S|$ is the number of samples in the parent node,
- $|S_i|$ is the number of samples in the i^{th} child node,

- p_k is the proportion of samples belonging to class k in node S ,
- Choose an optimal value of the threshold and provide justification in the report.
- Suppose you split across the 3rd dimension, resulting in two regions. Perform another split by repeating the above step. Once this split is performed, the entire p -dimensional space is divided into three regions.
- **Prediction & Evaluation [CO3]:** For each test sample from the three classes, traverse the tree to determine which leaf region the sample falls into, and **assign it the majority class label of the training samples in that region**. Report the following:
 - Overall classification accuracy on the test set.
 - Class-wise accuracy for each of the three classes.

Bagging and Out-of-Bag Error [CO3]

- Create 5 bootstrap datasets by sampling with replacement from the training data, each of size equal to the training set. Learn trees for all these datasets. For each tree, find the samples that were not used for training that tree and compute the Out-of-Bag (OOB) error. Report the average OOB error across all trees. [0.75]
- For test samples, use majority voting from all trees and report the total accuracy and class-wise accuracy. In case of a tie, you may choose any label.

Random Forest

- Use the same 5 bootstrapped datasets created in the above part. Train decision trees for all the datasets, but this time randomly select only k features out of p at every split. Choose the best split among only those k features. Repeat this process to build a tree with maximum height 2, as done in Part 1 of Question 2. [0.5] [CO3]
- Choose an optimal value of k and provide justification in the report.
- Train the trees and compute the OOB error. Report the test accuracy and OOB error. [0.25]
- Is this approach better than the previous one? If yes, explain how Random Forest improves performance. If not, explain why.

Question 3

Use the <https://www.kaggle.com/datasets/zalando-research/fashionmnist/data> Fashion-MNIST dataset for this question. The training set contains 60,000 samples and the test set contains 10,000 samples. Similar to MNIST, use only the classes 0, 1, and 2 (T-shirt/top,

Trouser, and Pullover), respectively. The preprocessing steps remain the same as for MNIST: flatten the images, normalize the pixel values, and then apply PCA with $p = 10$. [2.5]

- Learn a regression model using a **decision stump** (a decision tree with only one split) on the training set. Sort the values of x in ascending order. The candidate split thresholds should be the midpoints between consecutive sorted values. [CO2]
- For each candidate split, divide the data into left and right regions and compute the **Sum of Squared Residuals (SSR)**:

$$SSR = \sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2$$

where $\bar{y}_L$ and $\bar{y}_R$ are the mean response values in the left and right regions, respectively. [CO2]

- Choose the split that minimizes SSR. Denote this model as $h_1(x)$. The prediction for any sample should be the mean response value of the region to which the sample belongs. Report the Mean Squared Error (MSE) on the test set.

Bagging and Out-of-Bag Error [CO3]

- Create 5 bootstrap datasets by sampling with replacement from the training data, each of size equal to the training set.
- Train a regression decision stump on each bootstrap dataset and denote the models as $h_i(x)$. The bagging prediction should be the average of predictions from all models:

$$\hat{y}(x) = \frac{1}{5} \sum_{i=1}^5 h_i(x)$$

- For each model, compute the Out-of-Bag (OOB) error using Mean Squared Error. Report the average OOB error across all models. Report the test set MSE using bagging.
- Compare the performance of a single decision stump and bagging by plotting the predictions of the single decision stump and the bagged model on the same plot, along with the true function. Explain the difference observed. [CO1]